ОТЧЕТ ПО
ЛАБОРАТОРНОЙ РАБОТЕ №3

Выполнил:
Батуров Максим Дмитриевич
Группа: 6203-010302D

Оглавление

Задание №1 ..... **2**

Задание №2 ..... **2**

Задание №3 ..... **3**

Задание №4 ..... **4**

Задание №5 ..... **6**

Задание №6 ..... **7**

Задание №7 ..... **7**

Задание №1

Были изучены и проанализированы классы исключений стандартной библиотеки Java, включая Exception, IndexOutOfBoundsException, ArrayIndexOutOfBoundsException, IllegalArgumentException и IllegalStateException. Проведен анализ их иерархии, областей применения и рекомендуемых сценариев использования.

Задание №2

Разработаны два специализированных класса для обработки исключительных ситуаций. FunctionPointIndexOutOfBoundsException расширяет функциональность IndexOutOfBoundsException и предназначен для обработки ситуаций выхода за пределы допустимых индексов при работе с точками функции. InappropriateFunctionPointException наследует от Exception и используется для обработки случаев нарушения логической целостности данных. Оба класса содержат конструкторы, позволяющие передавать детализированные сообщения об ошибках.

```
package functions;

public class FunctionPointIndexOutOfBoundsException extends IndexOutOfBoundsException {  31 usages

    public FunctionPointIndexOutOfBoundsException(String errorMessage) {// конструктор класса принимающий строку
        super(errorMessage);
    }

    public FunctionPointIndexOutOfBoundsException() { // конструктор без параметров  no usages
    }
}
```

```java
package functions;

public class InappropriateFunctionPointException extends Exception {  19 usages

    public InappropriateFunctionPointException(String errorMessage) {// конструктор принимающий строку
        super(errorMessage);
    }

    public InappropriateFunctionPointException() {//конструктор без параметров  no usages
    }
}
```

Задание №3

Проведена комплексная модернизация класса ArrayTabulatedFunction с внедрением системы валидации во все ключевые методы. Конструкторы класса теперь выполняют проверку входных параметров, включая корректность заданного интервала и минимальное количество точек. Методы работы с точками осуществляют контроль границ массива и сохранение целостности данных. Особое внимание уделено обеспечению сохранения упорядоченности точек при выполнении операций модификации.

```java
public class ArrayTabulatedFunction implements TabulatedFunction {  5 usages
    private FunctionPoint[] points;  29 usages
    private int pointsCount;  30 usages
    private static final double EPSILON = 1e-10;  10 usages

    // создает табулированную ф-цию с нулевыми значениями
    public ArrayTabulatedFunction(double leftX, double rightX, int pointsCount) {  2 usages
        if (leftX >= rightX - EPSILON) {
            throw new IllegalArgumentException("Левая граница должна быть меньше правой");
        }
        if (pointsCount < 2) {
            throw new IllegalArgumentException("Количество точек должно быть не менее 2");
        }

        this.pointsCount = pointsCount;
        points = new FunctionPoint[pointsCount + 3];

        double step = (rightX - leftX) / (pointsCount - 1);

        for (int i = 0; i < pointsCount; i++) {
            points[i] = new FunctionPoint( x: leftX + step * i,  y: 0);
        }
    }
```

```java
    public void setPoint(int index, FunctionPoint point) throws FunctionPointIndexOutOfBoundsException, InappropriateFunctionPointException {
        if (index < 0 || index >= pointsCount) {
            throw new FunctionPointIndexOutOfBoundsException("Индекс " + index + " выходит за границы [0, " + (pointsCount - 1) + "]");
        }

        double newX = point.getX();

        // Проверка корректности новой координаты X
        if (index > 0 && newX <= points[index - 1].getX() + EPSILON) {
            throw new InappropriateFunctionPointException("Новая координата X должна быть больше X предыдущей точки");
        }
        if (index < pointsCount - 1 && newX >= points[index + 1].getX() - EPSILON) {
            throw new InappropriateFunctionPointException("Новая координата X должна быть меньше X следующей точки");
        }

        points[index] = new FunctionPoint(point);
    }
```

```java
public void setPointX(int index, double x) throws FunctionPointIndexOutOfBoundsException, InappropriateFunctionPointException {
    setPoint(index, new FunctionPoint(x, points[index].getY()));
}
```

Задание №4

При реализации класса LinkedListTabulatedFunction была применена архитектура двусвязного циклического списка с выделенным головным элементом. Внутренний класс FunctionNode реализован как статический с приватным модификатором доступа. Каждый узел содержит объект FunctionPoint для хранения данных и ссылки на соседние узлы. Основной класс включает поле head и счетчик элементов pointcount. Методы getNodeByIndex, addNodeToTail, addNodeByIndex и deleteNodeByIndex обеспечивают эффективное управление структурой данных.

```java
public class LinkedListTabulatedFunction implements TabulatedFunction {
    private FunctionNode head;  20 usages
    private int pointcount;  22 usages

    private static class FunctionNode {  25 usages
        private FunctionPoint point;  3 usages
        private FunctionNode prev;  3 usages
        private FunctionNode next;  3 usages

        public FunctionNode(FunctionPoint point) {  3 usages
            this.point = point; // устанавливаем точку
            this.prev = null; // предыдущий узел пока null
            this.next = null; // следующий узел пока null
        }

        public FunctionPoint getPoint() {  17 usages
            return point; // возвращаем точку
        }

        public void setPoint(FunctionPoint point) {  2 usages
            this.point = point; // устанавливаем новую точку
        }

        public FunctionNode getPrev() {  6 usages
            return prev; // возвращаем предыдущий узел
        }

        public void setPrev(FunctionNode prev) {  6 usages
            this.prev = prev; // устанавливаем предыдущий узел
        }

        public FunctionNode getNext() {  13 usages
            return next; // возвращаем следующий узел
        }

        public void setNext(FunctionNode next) {  6 usages
            this.next = next; // устанавливаем следующий узел
        }
    }
```

```java
private FunctionNode addNodeToTail(FunctionPoint point) {  3 usages
    FunctionNode newNode = new FunctionNode(point); // создаем новый узел
    FunctionNode tail = head.getPrev(); // получаем хвост списка

    newNode.setPrev(tail);
    newNode.setNext(head);
    tail.setNext(newNode);
    head.setPrev(newNode);

    pointcount++;
    return newNode; // возвращаем новый узел
}

private FunctionNode addNodeByIndex(int index, FunctionPoint point) {  1 usage
    if (index < 0 || index > pointcount) {
        throw new FunctionPointIndexOutOfBoundsException("индекс выходит за границы");
    }

    if (index == pointcount) {
        return addNodeToTail(point); // добавляем в конец
    }

    FunctionNode currentNode = getNodeByIndex(index); // получаем текущий узел
    FunctionNode newNode = new FunctionNode(point); // создаем новый узел
    FunctionNode prevNode = currentNode.getPrev(); // получаем предыдущий узел

    newNode.setPrev(prevNode);
    newNode.setNext(currentNode);
    prevNode.setNext(newNode);
    currentNode.setPrev(newNode);

    pointcount++;
    return newNode; // возвращаем новый узел
}

private FunctionNode deleteNodeByIndex(int index) {  1 usage
    if (index < 0 || index >= pointcount) {
        throw new FunctionPointIndexOutOfBoundsException("индекс выходит за границы");
    }

    FunctionNode nodeToDelete = getNodeByIndex(index); // узел для удаления
    FunctionNode prevNode = nodeToDelete.getPrev(); // предыдущий узел
    FunctionNode nextNode = nodeToDelete.getNext(); // следующий узел

    prevNode.setNext(nextNode);
    nextNode.setPrev(prevNode);

    pointcount--;
    return nodeToDelete; // возвращаем удаленный узел
}
```

Задание №5

В классе LinkedListTabulatedFunction полностью реализованы все методы, объявленные в интерфейсе TabulatedFunction. Конструкторы обеспечивают создание функции с равномерным распределением точек или заданными значениями. Все публичные методы используют внутренние операции работы со списком. Система обработки исключений полностью соответствует реализации в ArrayTabulatedFunction. Оптимизированы методы доступа к элементам списка за счет использования двусвязной структуры.

```
package functions;

public interface TabulatedFunction {  9 usages  2 implementations
    // получить левую границу обл. опр.
    double getLeftDomainBorder();  3 usages  2 implementations

    // получить правую границу обл. опр.
    double getRightDomainBorder();  3 usages  2 implementations

    // вычислить значение ф-ции в заданной точке
    double getFunctionValue(double x);  2 usages  2 implementations

    // получить количество точек в ф-ции
    int getPointsCount();  1 usage  2 implementations

    // получить точку по индексу
    FunctionPoint getPoint(int index) throws FunctionPointIndexOutOfBoundsException;  2 usages  2 implementations

    // установить точку по индексу
    void setPoint(int index, FunctionPoint point) throws FunctionPointIndexOutOfBoundsException, InappropriateFunctionPointException;

    // получить координату x по индексу
    double getPointX(int index) throws FunctionPointIndexOutOfBoundsException;  1 usage  2 implementations

    // установить координату x по индексу
    void setPointX(int index, double x) throws FunctionPointIndexOutOfBoundsException, InappropriateFunctionPointException;  1 usage  2 im

    // получить координату y по индексу
    double getPointY(int index) throws FunctionPointIndexOutOfBoundsException;  1 usage  2 implementations

    // установить координату y по индексу
    void setPointY(int index, double y) throws FunctionPointIndexOutOfBoundsException;  1 usage  2 implementations

    // удалить точку по индексу
    void deletePoint(int index) throws FunctionPointIndexOutOfBoundsException, IllegalStateException;  2 usages  2 implementations

    // добавить новую точку
    void addPoint(FunctionPoint point) throws InappropriateFunctionPointException;  2 usages  2 implementations
}
```

Задание №6

Проведен рефакторинг архитектуры проекта с выделением общего интерфейса TabulatedFunction. Существующий класс TabulatedFunction был переименован в ArrayTabulatedFunction. Обе реализации теперь имплементируют единый интерфейс, что обеспечивает возможность их полиморфного использования. Интерфейс содержит объявления всех основных операций работы с табулированными функциями, включая методы доступа к точкам, модификации данных и вычисления значений.

Задание №7

Разработана комплексная система тестирования в классе Main. Тестовые сценарии охватывают все основные аспекты работы. Специализированный метод testExceptions() проверяет корректность работы системы обработки исключений. Демонстрируется принцип полиморфизма через использование интерфейсного типа для работы с различными реализациями.

Пример работы:

```
"C:\Program Files\Java\jdk-25\bin\java.exe" "-javaagent:C:\Program Files\JetB
Тестирование ArrayTabulatedFunction:
Область определения: 1.0 - 5.0
Список точек:
0: x=1.0 y=2.0
1: x=2.0 y=5.0
2: x=3.0 y=10.0
3: x=4.0 y=17.0
4: x=5.0 y=26.0
f(3.2) = 11.400000000000002
f(0) = NaN
После смены Y:
Список точек:
0: x=1.0 y=2.0
1: x=2.0 y=7.0
2: x=3.0 y=10.0
3: x=4.0 y=17.0
4: x=5.0 y=26.0
После удаления:
Список точек:
0: x=1.0 y=2.0
1: x=2.0 y=7.0
2: x=4.0 y=17.0
3: x=5.0 y=26.0
После добавления:
Список точек:
0: x=1.0 y=2.0
1: x=2.0 y=7.0
2: x=2.8 y=15.0
3: x=4.0 y=17.0
4: x=5.0 y=26.0

Тестирование LinkedListTabulatedFunction:
Область определения: 1.0 - 5.0
Список точек:
```

```
Тестирование LinkedListTabulatedFunction:
Область определения: 1.0 - 5.0
Список точек:
0: x=1.0 y=2.0
1: x=2.0 y=5.0
2: x=3.0 y=10.0
3: x=4.0 y=17.0
4: x=5.0 y=26.0
f(3.2) = 11.400000000000002
f(0) = NaN
После смены Y:
Список точек:
0: x=1.0 y=2.0
1: x=2.0 y=7.0
2: x=3.0 y=10.0
3: x=4.0 y=17.0
4: x=5.0 y=26.0
После удаления:
Список точек:
0: x=1.0 y=2.0
1: x=2.0 y=7.0
2: x=4.0 y=17.0
3: x=5.0 y=26.0
После добавления:
Список точек:
0: x=1.0 y=2.0
1: x=2.0 y=7.0
2: x=2.8 y=15.0
3: x=4.0 y=17.0
4: x=5.0 y=26.0

Проверка исключений:
ArrayTabulatedFunction: Левая граница должна быть меньше правой
LinkedListTabulatedFunction: некорректные границы области определения
ArrayTabulatedFunction: Количество точек должно быть не менее 2
Array getPoint(-1): Индекс -1 выходит за границы [0, 4]
LinkedList getPoint(10): индекс выходит за границы
Array setPoint: Новая координата X должна быть больше X предыдущей точки
LinkedList setPointX: нарушение порядка точек
Array addPoint: Точка с X = 2.0 уже существует
Array deletePoint: Невозможно удалить точку - функция должна иметь как минимум 2 точки
```